
Java™

Page :
Date : / /

* Array :

Q.1] What do you mean by an Array ?

⇒ Array is a collection of similar type of elements which has contiguous memory location.

Q.2] How to create an Array ?

⇒ To create an array with default values, you first declare the array variable using the syntax datatype[] arrayName = new datatype[length], where datatype is the type of data the array will hold, arrayName is the name of the array, and length is the number of elements the array will contain.

Example : int[] numbers = new int[5];

Q.3] Can we change the size of an array at run time ?

⇒ Thus the size of the array is determined at the time of its creation or, initialization once it is done you cannot change the size of the array.

Q.4] Can you declare an array without assig-ning the size of an array ?

⇒ No, you cannot declare an array without specifying its size or initializing it with

Page :
Date : / /

elements. when you declare an array, you need to provide either the size or initialize it with values.

Q.5] what is the default value in Array?

⇒ when you declare an array of primitive data types (e.g. int, float, boolean, etc.) the elements of the array are initialized to their default values.

Q.6] what is a 1D array with an example?

⇒ A 1-D array can be visualized as a single row or a column of array elements that are represented by a variable name and whose elements are accessed by index values.

example :-

```
Public class Demo {
    public static void main (String [] args) {
        int [] myArray = new int [5];
        myArray [0] = 10;
        myArray [1] = 20;
        myArray [2] = 30;
        myArray [3] = 40;
        myArray [4] = 50;

System.out.println ("Element at index 0:" +
    myArray [0]);
```

```
System.out.println("Element at index 2 : " +
    myArray[2]);
int length = myArray.length;
for(int i=0; i<length; i++){
    system.out.println("Element at index" +
i + " : " + myArray[i]);
    }
  }
}
```

output :-

```
Element at index 0 : 10
Element at index 2 : 30
Element at index 0 : 10
Element at index 1 : 20
Element at index 2 : 30
Element at index 3 : 40
Element at index 4 : 50
```

Q.7] write a program on a 2D array?

⇒

```
Class TwoDArray{
  public static void main(String[] args){
    int[][] matrix = {
        {1,2,3},
        {4,5,6},
        {7,8,9}
    };
    int numRows = matrix.length;
    int numCols = matrix[0].length;
```

```
System.out.println("Number of rows : "+
                                numRows);
System.out.println("Number of columns : "+
                                numCols);
System.out.println("Elements of the 2D
  array : ");
for(int row=0; row<numRows; row++){
   for(int col=0; col<numCols; col++){
      System.out.print(matrix[row][col]+
                                " ");}
      system.out.println();
   }
}
```

output :

```
Number of rows : 3
Number of columns : 3
Elements of the 2D array :
 1  2  3
 4  5  6
 7  8  9
```

* Exception Handling

Page :
Date : / /

Q.1] Explain different types of errors in java.

⇒ In any programming language we categorise into 2 types
1. Syntax Error / Compile Time Mistakes
2. Logical Error / RunTimeMistakes

- **Syntax error / compileTime Mistakes :**
  - It refers to the mistake done by the programmer with respect to syntax.
  - These mistakes are identified by the compiler so we say it as "compileTimeMistake".

- **Logical Error / RunTimeMistakes :**
  - It refers to the mistakes done by the programmer in terms of writing ~~a~~ logic
  - These mistake are identified by JVM during the execution of a program, so we say it as "RunTimeMistakes".

Q.2] What is an Exception in Java.

⇒ An unwanted / expected event that disturbs the normal flow of execution of a program is called "Exception handling".
- The main objective of exception handling is to handle the exception.
- It is available for graceful termination of program.

Page :
Date : / /

Q.3] How can you handle exception in java? Explain with an example.

⇒ Exception handling can be performed using.

Try : The set of statements or code which requires monitoring for an example. exception is kept under this block.

Catch : this block catches all exceptions that were trapped in the try block

Finally : this block is always performed irrespective of the catching of exceptions in the try or catch block.

Ex :

```
Class Demo{
    public static void main (String args[]){
        try {
            System.out.println ("Hello-" + " "
                                + 1/0);
        }
        Catch(ArithmeticException e)
        {
        System.out.print(" World ");
        }
    }
}
```

Page :
Date : / /

Q.4] Why do we need exception handling in Java ?

⇒ If there is no try and catch block while an exception occurs, the program will terminate. Exception handling ensures the smooth running of a program without program termination.

Q.5] What is the difference between exception and error in java ?

⇒ Error typically happen while an application is running. For instance, out of memory error occurs in case the JVM runs out of memory. On the other hand, exceptions are mainly caused by the application, For instance, null pointer exception happens when an app tries to get through a null object.

Q.6] Name the different types of exception in Java.

⇒ Based on handling by JVM, there are typically two types of exception in Java:
Checked : occur during the compilation. Here, the compiler checks whether the exception is handled and throws an error accordingly.
Unchecked :
occur during program execution. These are not detectable during the compilation process.

Page :
Date : / /

Q.7] Can we just use try instead of finally and catch blocks?

⇒ No, doing so will show a compilation error. catch or finally block must always accompany try block. We can remove either finally block or catch block, but never both.

* Collection Framework

Page :
Date : / /

Q.1] What is the Collection Framework in java ?

⇒ Collection Framework is a combination of classes and interface, which is used to store and manipulation the data in the form of objects. It provides various classes such as ArrayList, Vector, Stack and Hashset, etc. and interfaces such as List, Queue, set, etc. for this purpose.

Q.2] What is difference between ArrayList and LinkedList ?

⇒

ArrayList	LinkedList
① ArrayList uses a dynamic array.	① LinkedList uses a doubly linkedlist.
② ArrayList is not effi-cient for manipulation because too much is required.	② Linked list is efficient for manipulation.
③ ArrayList provides random access.	③ LinkedList is does not provide random access.
④ ArrayList takes less memory overhead as it stores only object.	④ LinkedList takes more memory overhead as it stores the object as well as the addin-ess of that object.

Page :
Date : / /

Q.3] What is the difference between Iterator and ListIterator?

⇒

Iterator	ListIterator
(1) The Iterator traverses the elements in the forward direction only.	(1) ListIterator traverses the elements in backward and forward directions both.
(2) The iterator can be used in list, set, and Queue	(2) ListIterator can be used in list only.
(3) The Iterator can only perform a remove operation while traversing the collection.	(3) ListIterator can perform ? add, ? remove ? and ? set? operation while traversing the collection.

Q.4] What is the difference between Iterator and Enumeration ?

Iterator	Enumeration
The Iterator can traverse legacy and non-legacy elements	Enumeration can traverse only legacy elements.
The Iterator is fail-fast.	Enumeration is not fail-fast.

Page :
Date : / /

The Iterator is slower than Enumeration.	Enumeration is faster than Iterator.
The Iterator can perform a remove operation while traversing the collection.	The Enumeration can perform only traverse operations on the collection.

Q.5] What is the difference between List and set.

⇒ The list and set both extend the collection interface. However, there are some differences between the two which are listed below.

- The list is an ordered collection which which maintains the insertion order whereas set is an unordered collection which does not preserve the insertion order.
- The list interface contains a single legacy class which is vector class whereas the set interface does not have any legacy class.
- The list interface can allow a number of null values whereas set interface only allows a single null value.
- The list can contain duplicate elements whereas set neglects unique items.

Page :
Date : / /

Q.6] What is the difference between Hashset and Treeset ?

⇒

Both Hashset and Treeset are implementations of the set interface in java, but they have some difference in the terms of their properties and usage:

- **ordering :**
  Hashset is an unordered collection of elements, while Treeset is a stored set of elements based on their natural order or a custom comparator.
- **Duplication :**
  Hashset does not allow duplicate elements, while Treeset does not allow duplicates as well.
- **Implementation :**
  Hashset is implemented using a hash table, while Treeset is implemented using a self-balancing binary search tree (Red-Black tree)
- **Performance :**
  Hashset has constant-time complexity O(1) for adding, removing and testing the existence of an element, while Treeset has a logarithmic-time complexity O(log n) for these operation due to the self-balancing property.

Page :
Date : / /

• Memory usage :

Hashset uses less memory than Treeset because it only stores the elements, while Treeset stores additional information for maintaining the order.

• Iteration :

Hashset provides no guarantees regarding the order of iteration, while Treeset guarantees the elements are iterated in sorted order.

• Usage :

Hashset is suitable when ordering is not important, and fast access and membership tests are needed. Treeset is suitable when elements need to be sorted or accessed in a specific order.

Q.7] What is the difference between Array and ArrayList?

⇒

Both arrays and ArrayList are used to store collections of elements in java, but they have some difference in terms of their properties and usage:

• Type :

Arrays can store elements of primitive data types as well as objects, while ArrayList can only store objects.

• Size :

The size of an array is fixed once it is created, while the size of an

Page :
Date : / /

ArrayList can be dynamically increased or decreased by adding or removing elements.

- Mutability :-

Arrays are mutable, meaning that you can modify the elements in an array after it has been created. ArrayList is also mutable, but the only way to modify it is by adding, removing or modifying elements.

- Methods :-

Arrays have a limited set of methods compared to ArrayList, which provides more methods for manipulating the collection, such as adding, removing, and sorting elements.

- Initialization :-

Arrays can be initialized with values at the time of creation, while ArrayList requires the use of methods to add elements to the collection.

* Multithreading

Q.1] what do you mean by Multithreading ? why is it important ?

⇒ Multithreading means multiple threads and is considered one of the most important features of java. As the name suggests, it is the ability of a CPU to execute multiple threads independently at the same time but share the process resources simultaneously.

Its main purpose is to provide simultaneous execution of multiple threads to utilize the the CPU time as mush as possible. It is a java feature where one can subdivide the specific program into two or more threads to make the execution of the program fast and easy.

Q.2] what are the benefits of using multi-threading ?

⇒ There are various benefits of multi-threading as given below:

- Allows to write effective program that utilize maximum CPU time.
- Saves time and parallelism tasks.
- In an exception occurs in a single thread, it will not affect other threads as threads are independent.
- Improve performance as compared to

Page :
Date : / /

traditional parallel programs that use multiple processes.
- Allow the program to run continuously even if a part of it is blocked.
- Less resource-intensive than executing than a multiple processes at the same time.
- Increase use of CPU resources and reduce costs of maintenance.

Q.3] What is Thread in java?

⇒

A Thread is a very light-weighted process, or we can say the smallest part of the process that allows a program to operate more efficiently by running multiple tasks simultaneously.

Q.4] What are the two ways of implementing thread in java?

⇒

There are basically two ways of implementing thread in java as given below:

Extending the Thread class:

Example:

```
class Demo extends Thread
{
  public void run()
  {
```

```
    System.out.println(" My thread is in running state.");
}
public static void main (String args []) {
    Demo obj = new Demo ();
    obj.start();
    }
}
```

output:
My thread is in running state.

Implementing Runnable interface in java:

```
class Demo implents Runnable
{
  public void run()
  {
    System.out.println("My thread is in running state.");
}
  public static void main (String args[])
{
    Demo obj=new Demo();
    obj.start();
  }
}
```

output :-
My thread is in running state.

Page :
Date : / /

Q.5] What's the difference between thread and process ?

⇒

Process	Thread
program in Execution	seperate path of execution one or more threads is called as process.
Generally slower than thread.	Generally faster than process.
Allocate new resource each time we run a program	Share code, heap, data and except stack - share resource of process

Q.6] How can we create daemon threads?

⇒ we can create daemon threads in java using the thread class setDaemon(true). It is used to mark the current thread as daemon thread or user thread. isDaemon() method is generally used to check whether the current thread is daemon or not. If the thread is a daemon, it will return true otherwise it returns false.

Page :
Date : / /

Example :

```
public class DemonThread extend Thread
{
   public DaemonThread (String name) {
         super (name);
   }
   public void run ()
   {
      if (Thread.currentThread ().isDaemon())
      {
         System.out.println (getName() + "
is Daemon thread");
      }
      else
      {
         System.out.println (getName () + "
is User thread");
      }
   }
   public static void main (String args[])
   {
      DaemonThread t1 = new DaemonThread
         ("t1");
      DaemonThread t2=new DaemonThread ("t2")
      DaemonThread t3=new DaemonThread ("t3")
      t1.start ();
      t2.start ();
      t3.setDaemon (true);
      t3.start ();
   }
}
```

Page :
Date : / /

Output :

t1 is Daemon thread
t3 is Daemon thread
t2 is Daemon thread

Q.7] What are the wait() and sleep() methods ?

⇒

sleep() : This method is used to pause the execution of current thread for a specified time in milliseconds. Here, Thread does not lose its ownership of the monitor and resumes its execution

Example :
Synchronized (monitor) {
Thread.sleep(1000); Here lock Is Held By The current Thread
}

wait() : This method is defined in object class. It tells the calling thread to wait until another thread invokes the notify() or notifyAll() method for this object, The thread waits until it reobtains the ownership of the monitor and Resumes execution.
Example :
Synchronized (monitor){
monitor.wait() Here LOCK IS Released By current Thread
}

* Map and Generics

Page :
Date : / /

Q.1] What is a Map in Java?

⇒

A map is a collection in Java that stores data as key-value pairs, where each key is unique.

Q.2] What are the commonly used implementation of map in java?

⇒

The commonly used implementation of map in Java are HashMap, TreeMap, LinkedHashMap and ConcurrentHashMap.

Q.3] What is the difference between HashMap and TreeMap?

⇒

HashMap	TreeMap
(1) any order for elements.	(1) It provides orders for elements.
(2) Its speed is fast.	(2) Its speed is slow
(3) It has only basic features	(3) It has advanced features.
(4) It's complexity is O(1)	(4) Its complexity is O(log n).

Page :
Date : / /

Q. ④ How do you check if a key exists in a map in java?

⇒

We can check if a key exists in a map in java using the containsKey() method at the get() method. The containsKey() method returns a boolean value indicating whether the map contains the specified key, while the get() method returns the value associated with the specified key, or null if the key is not present in the map.

Q ⑤. What are Generics in java?

⇒

Generics in java are used to provide type safety and reduce code redundancy by allowing the use of generic types. It allows classes, methods, and interfaces to be written generically, without specifying the type of data being used.

Q.6] What are the benefits of using Generics in java?

⇒ The benefits of using Generics in java are:

Type safety
code reusability
improved readability
Reduced code redundancy
Improved performance.

Page :
Date : / /

Q.7] What is a Generic class in Java?

⇒ A Generic class in Java is a class that can work with different types of data. It is defined using a type parameter enclosed in angle brackets, which up represents the type of data being used.

Q.8] What is a type parameter in Java Generics?

⇒ A Type parameter in Java Generics is a placeholder for the type of data that is used by a generic class method. It is defined using a single uppercase letter enclosed in angle brackets, such as <T> or <E>.

Q.9] What is a Generic method in Java?

⇒ A Generic method in Java is a method that can work with different types of data. It is defined using a parameter enclosed in angle brackets, which represents the type of data being used.

Q.10] What is the difference between ArrayList and ArrayList<T>?

⇒ ArrayList is a non-generic class, while ArrayList<T> is a generic class. ArrayList<T> provides type safety.

Page :
Date : / /

* Java variables and Data types

Q.1] What is statically typed and dynamically typed programming language?

⇒ Dynamically-typed languages perform type checking at runtimes while statically typed languages perform type checking at compile time.

Q.2] What is the variable in Java?

⇒ Variable are containers for storing data values.

Every variable in Java is assigned a data type that designates the type and quantity of value it can hold.

Q.3] How To Assign a value To variable?

⇒ The first time a variable is assigned a value, it is said to be initialised.

Ex.

```
import java.io.*;
Class Demo{
    Public static void main(String args[])
    {
        int a=5; // initialising variables
                    directly
        System.out.println("The value
of a is: "+a); // printing value of a
    }
}
```

output:

The value of a is : 5

Page :
Date : / /

Q.4] What are primitive Data types in Java?

⇒ In Java, the primitive data types are the predefined data types of Java. They specify the size and type of any standard values.

Java has 8 primitive data types namely byte, short, int, long, float, double, char and boolean.

Q.5] What are the identifiers in Java?

⇒ In Java, identifiers are used for identification purposes.

Java identifiers can be a class name, method name, variable name, or label.

Q.6] List the operators in Java?

⇒ Operators are symbols that perform operations on variables and values. Here's a list of some common operators in Java:

(1) Arithmetic operators: They are used to perform simple arithmetic operations on primitive data types.

- * : Multiplication
- / : Divison
- % : Modulo
- + : Addition
- − : Subtraction.

Page :
Date : / /

(2) Comparison operators :
- == (equal to)
- != (not equal to)
- < (less than)
- > (greater than)
- <= (less than or equal to)
- >= (greater than or equal to)

(3) Assignment operators :
- = (assignment)
- += (addition assignment)
- -= (subtraction assignment)
- *= (multiplication assignment)
- /= (division assignment)
- %= (modulus assignment)

(4) Logical operators :
- && (logical AND)
- || (logical OR)
- ! (logical NOT)

(5) Bitwise operators :
- & (bitwise AND)
- | (bitwise OR)
- ^ (bitwise XOR)
- ~ (bitwise NOT)
- << (left shift)
- >> (right shift)
- >>> (unsigned right shift)

Page :
Date : / /

(6) Increment / Decrement operations :
- ++ (increment by 1)
- -- (decrement by 1)

(7) Conditional (Ternary) operators :
- ? : (conditional operator)

(8) Instanceof operator :
- instanceof (used for type checking)

Q.7] Explain about increment and Decrement operators and given an example.

⇒

The increment operator (++) add 1 to the operator value contained in the variable. The decrement operator (--) substract from the value contained in the variable.
- There are two ways of representing increment and decrement operators.

(1) as a prefix i.e. operator precedes the operand. Eg. ++i; --j;

(2) as a postfix i.e. operator follows the operand. Eg. i--; j++;

Example :

```
import java.util.*;
class Demo {
    public static void main (String args[]) {
        int a;
        Scanner scan = new Scanner (System.in);
```

```
System.out.println ("Enter the value of a :");
    a = scan.nextInt();
    System.out.println ("Before increment
A :" +a );
    System.out.println ("After Increment A:" +a);
    a++;
        System.out.println ("After two Time
Increment A: " + a);
            System.out.println ("Before Decremen
A : "+a);
        --a;
        a--;
            System.out.println ("After Decremen
Two Time A : " +a);
    }
}
```

output :

```
Enter value of a : 10
    The
Before Increment A : 10
After Increment A : 11
After Two Time Increment A : 12
Before Decrement A : 12
After Decrement Two Time A : 10
```

Page :
Date : / /

* Operators and Loops :

Q.1] What are the conditional operators in Java ?

⇒ In Java, conditional operators check the condition and decides the desired result on the basis of both conditions. In this section, we will discuss the conditional operator in Java.

Q.2] What are the types of operators based on the number of operands ?

⇒ There are two types of operators : unary and binary. Unary operators perform an action with a single operand. Binary operators perform actions with two operands. In a complex expression, (two or more operands) the order of evaluation depends on precedence rules.

Q.3] What is the use of switch case in Java programming ?

⇒ The switch case in Java is used to select one of many code blocks for execution.

Break keyword :

As Java reaches a break keyword, the control breaks out of the switch block. The execution of code stops on encountering this keyword, and the case testing inside the block ends as the match is found.

Page :
Date : / /

Default keyword : The keyword is used to specify the code executed when the expression code does not match any test case.

Q.4] what are the priority levels of arithmetic operation in java ?

⇒

Arithmetic operations follow the standard order of operations, also known as the BODMAS/BIDMAS rule.

The priority level of arithmetic operations is as follows, from highest to lowest:

(1) Parentheses : operations within parentheses are evaluated first.
(2) Exponentiation (^) : If used, exponentiation is evaluated next.
(3) Multiplication (*) and Division (/) : multiplication and division are evaluated from left to right.
(4) Addition (+) and subtraction (-) : Addition and subtraction are evaluated from left or to right.

Q.5] what are the conditional statements and use of conditional statements in java ?

⇒ you can use these conditions to perform different actions for different decisions.

Page :
Date : / /

Java has the following conditional statements:
use if to specify a block of code to be executed, if a specified condition is true. use else to specify a block of code to be executed, if the same condition is false.

Q.6] What is the syntax of if else statement.

⇒

Syntax:

```
if (condition){
// code to be executed
}
```

Q.7] What are the 3 types of iterative statements in Java?

⇒

- For loop : Executes a set of statements a fixed number of times.
- While loop : Repeats a set of statements at given condition is true.
- Do-while loop : Repeats a set of statement at least once and then continues to repeat as long as a given condition is true.

Page :
Date : / /

Q.8] write the difference between for loop and do-while loop ?

⇒

For loop.	Do-while loop
(1) Initialization is done before the loop is executed.	(1) Initialization is done after the loop is executed.
(2) The condition is checked before each iteration for the loop.	(2) The condition is checked after each iteration of the loop.
(3) The loop terminates terminates when the condition is no longer met.	(3) The loop terminate when the condition is first met.

Q.9] write a program to print numbers from 1 to 10.

⇒

```
Public class Demo{
    public static void main (String args[]){
        for(int i=1; i<=10; i++){
            System.out.println(i);
        }
    }
}
```

Page :
Date : / /

* String in Java

Q.1] What is String in Java ?

⇒

String is a sequence of characters. a string is an object that represents a number of character values. For example : "hello" is a string containing a sequence of character 'h', 'e', 'l', 'l' and 'o'.

Q.2] Types of string in java are ?

⇒

∴ There are two types of string in Java :

① primitive strings :

These are string literals or string calls from a non-constructor context. A constructor is a special method used to initialize objects.

② Object strings :

These are strings created using the new operator. object strings create two objects, whereas primitives create just one.

Object strings create the string literal and the variable to refer to it.

Q.3] In how many ways can you create strings objects in java ?

⇒ There are two ways to create a string object in java :

Page :
Date : / /

- using a String literal :

A String literal is sequence of characters enclosed in double quotes.
For example :

String s = "Hello, World!";

- using the new ~~Str~~ keyword :

The new keyword is used to create a new object of a specific class.
For example :

String s = new String ("Hello, World!");

Q.4] What is a string constant pool ?

⇒ In Java, Strings are the type of objects that can store the characters or values. A String acts the same as an array of characters in Java.

Memory allocation is not possible without string constant pool. In this article, we will learn about Java String constant pool.

Q.5] What do you mean by mutable and immutable objects ?

⇒ Mutable objects are those whose state can be changed after they have been created, while immutable objects are those whose state cannot be

changed after they have been created.

Q.6] Where exactly is the String constant pool located in the memory?

⇒ The Java String constant pool is located in the heap memory of the Java Virtual machine.